En este paso, es fundamental definir el algoritmo de Machine Learning que utilizaremos para clasificar los textos. Esto implica transformar los textos en diversas representaciones y luego entrenar el modelo con los datos extraídos de nuestro DataFrame. Para evaluar la efectividad de cada método empleado, incorporaremos la métrica F1-score, que mide el equilibrio entre precisión y exhaustividad (recall). Esta métrica es especialmente útil en casos donde las clases están desbalanceadas o cuando los falsos positivos y negativos tienen un impacto relevante.

Dado que estamos más familiarizados con el algoritmo Random Forest, hemos decidido emplear el módulo *RandomForestClassifier* (Algoritmo de aprendizaje supervisado que utiliza un conjunto de árboles de decisión) de *sklearn* como nuestro método de clasificación en esta implementación.

Hemos diseñado unas celdas para analizar diferentes aspectos de los hiper parámetros del modelo RandomForest y evaluar cuáles proporcionan el mejor rendimiento. En el notebook “implementacion_modulo_2.ipynb” están algunas de estas celdas.

Es importante mencionar que la columna “subreddit” tiene valores faltantes, en concreto treinta, por lo que nuestros datos de entrada se reducirán en treinta filas. (“data.subreddit.isna().sum()”).

Por otra parte, también nos gustaría subrayar la utilización de una serie de métodos, que los aplicaremos en múltiples ocasiones durante la práctica, porque son claves para la ejecución de la misma. Provienen de la librería *sklearn* y son:

- “train_test_split”: Divide los datos en conjuntos de entrenamiento y prueba.
- “RandomForestClassifier”: Implementa el modelo de clasificación.
- “f1_score” y “clasification_report”: Para evaluar el modelo.

Para seguir el progreso de las implementaciones, debido a que algunos códigos tardan un poco en compilar, utilizamos en determinadas ocasiones la librería *tqdm*, que nos da la posibilidad de observar una barra de progreso que permite monitorear el avance en los procesos (muy útil en datasets grandes como sobre el que estamos trabajando).

- **Método TF-IDF**

División de datos (Entrenamiento y Prueba): “X_train, X_test, y_train, y_test = train_test_split(data["clean_post"], data["subreddit"], test_size=0.3, random_state=42)”. En esta línea se especifica que el 30% de los datos serán asignados al conjunto de prueba

(X_test e y_test), y el 70% al conjunto de entrenamiento (X_train e y_train). La división es aleatoria dentro del conjunto original, pero respetando la proporción especificada por *test_size*.

Vectorización TF-IDF: El parámetro "max_df=0.95" significa que se descartan las palabras que aparecen en más del 95% de los documentos, lo que ayuda a eliminar términos demasiado comunes que no aportan mucha información. El parámetro "min_df=5" asegura que solo se tengan en cuenta las palabras que aparecen al menos en 5 documentos, eliminando así las palabras demasiado raras.

Codificación de Etiquetas: Una vez divididos los datos, se utiliza *LabelEncoder* para convertir las etiquetas categóricas (los subreddits) en valores numéricos. Codificamos las etiquetas antes de entrenar el modelo, porque sino el clasificador no podrá procesarlas correctamente.

Entrenamiento del Modelo (Random Forest). Algunos de los parámetros clave son:

- *n_estimators=100*: El número de árboles de decisión. Elegimos 100 para lograr un buen balance entre rendimiento y precisión sin comprometer demasiado el tiempo de entrenamiento.

- *n_jobs=-1*: Utiliza todos los núcleos disponibles en el procesador para paralelizar el entrenamiento y acelerar el proceso, especialmente útil cuando se tienen grandes volúmenes de datos.

- *bootstrap=True*: Indica que el muestreo de los datos para entrenar cada árbol se hace con reemplazo (bootstrap). Esto ayuda a reducir el sobreajuste y mejora la generalización del modelo.

- *max_depth=50*: Limita la profundidad máxima de cada árbol. Al establecerlo en 50, se permite una mayor complejidad en la estructura de los árboles, pero se evita que se sobreajusten a los datos de entrenamiento.

- *min_samples_leaf=1*: Define el número mínimo de muestras necesarias en una hoja del árbol. Con valor de 1, se permite una gran flexibilidad, lo que puede capturar patrones complejos.

- *min_samples_split=2*: Define el número mínimo de muestras necesarias para dividir un nodo. Al configurarlo en 2, el modelo es más flexible y puede generar divisiones detalladas, lo que ayuda a modelar datos más complejos.

- *max_features=100*: Limita el número de características a considerar en cada división de árbol. Esto reduce la correlación entre los árboles, ayudando a evitar el sobreajuste.

Predicción y Evaluación: Una vez que el modelo está entrenado, se realiza la predicción en los datos de prueba ("X_test_tfidf"). A continuación, se evalúa el rendimiento del modelo utilizando el *F1 Score*, una métrica que es especialmente útil cuando hay un desequilibrio en las clases (como puede ser el caso con muchos subreddits).

- **Método Named-Entity Recognition**

Inicialmente, se consideraron varias opciones para extraer entidades nombradas, entre ellas *spaCy* y *Stanza*, que son herramientas avanzadas para el procesamiento de lenguaje natural y ofrecen modelos preentrenados para NER (Named-Entity Recognition). Sin embargo, al tratar de procesar un conjunto de datos de más de 270000 filas (como el nuestro), estas herramientas resultan poco viables debido a los altos costos computacionales que conllevan, especialmente cuando se trabaja con grandes volúmenes de datos.

Por estas razones, también se optó por un enfoque más simple y eficiente, basado en expresiones regulares, que permite identificar las entidades nombradas más relevantes de manera rápida y con menos recursos computacionales. Si bien este enfoque es más eficiente, reduce mucho la precisión de los resultados obtenidos.

A pesar de lo comentado con anterioridad, hemos dejado constancia de la opción de trabajar con *spaCy* para aquellos casos en los que se pueda reducir el tamaño del dataset, ya que, aunque esta opción será más precisa, su uso en grandes volúmenes de datos podría durar horas o incluso días de procesamiento. Por lo tanto, se ha decidido limitar el tamaño del conjunto de datos en el enfoque con *spaCy*, lo que mejora la precisión de los resultados, pero al mismo tiempo, no toma una muestra tan representativa del total de los datos. En cambio, las expresiones regulares se emplean en el caso de que se necesite trabajar con todos los datos, ya que es una opción más rápida, aunque menos precisa.

<u>Explicación del primer código</u>

El código que implementamos se basa en la extracción de entidades nombradas utilizando expresiones regulares. A continuación, se detalla el proceso general:

- Extracción de entidades: La función *extract_named_entities* utiliza varias expresiones regulares para identificar y extraer diferentes tipos de entidades del texto. Se definen tres patrones de búsqueda:

    - CamelCase: Detecta palabras que están en formato CamelCase (por ejemplo, "MachineLearning" o "DataScience").
    - Abreviaturas: Captura secuencias de letras mayúsculas que son abreviaciones (por ejemplo, "NASA" o "USA").
    - camelCase: Encuentra palabras en formato camelCase (por ejemplo, "firstName" o "userId").

    Estos patrones se combinan para identificar una variedad de posibles entidades dentro del texto. Posteriormente, las entidades extraídas se filtran según su longitud y se eliminan las que coinciden con *stopwords* (palabras vacías que no aportan valor semántico). Finalmente, las entidades filtradas se devuelven como una cadena de texto, separada por espacios.

- Vectorización de entidades: A continuación, utilizamos el *CountVectorizer* de *sklearn*, una herramienta que convierte las entidades extraídas en una representación numérica adecuada (vectores dispersos) para los algoritmos de clasificación.

En resumen, el uso de expresiones regulares ofrece una solución eficiente desde el punto de vista del tiempo de procesamiento, sacrificando algo de precisión en comparación con herramientas de NER más avanzadas como *spaCy* o *Stanza*. Este enfoque demuestra que, cuando el tiempo de procesamiento es una limitación importante, se pueden obtener resultados sin necesidad de recurrir a técnicas más complejas y lentas. Si bien *spaCy* ofrece una mayor precisión, su uso en grandes conjuntos de datos puede resultar inviable, por lo que se opta por trabajar con muestras más pequeñas en su caso.

- **Método Word Embeddings**

Hemos optado por utilizar modelos contextuales y semánticos (*context and semantic-aware*) basados en *Transformers* en lugar de *Word2Vec* debido a las significativas ventajas que ofrecen en la representación del lenguaje natural, como su flexibilidad y precisión; por esto mismo, es el método más utilizado en la actualidad. A continuación, explicamos el motivo de esta elección y cómo funcionan brevemente ambos enfoques:

Word2Vec utiliza embeddings estáticos: cada palabra tiene un único vector, sin importar el contexto en el que aparece. Este enfoque tiene las siguientes limitaciones:

- Falta de sensibilidad al contexto: Palabras polisémicas.
- Representaciones limitadas.

Ventajas de los modelos contextuales y semánticos:

DistilBERT es una versión optimizada de *BERT* (modelo de lenguaje que utiliza *Transformers*) que permite realizar tareas avanzadas de procesamiento de lenguaje natural de manera eficiente y con menores costos computacionales. Es ideal para aplicaciones donde el rendimiento en tiempo real y la eficiencia son importantes. Proporcionan embeddings dinámicos, adaptándose al contexto específico de cada palabra en una oración. Incluyen información semántica global, mejorando la comprensión del significado de frases completas.

Explicación del código

Aquí proponemos una explicación de algunas de las líneas que pueden generar mayor problemática a la hora de razonarlas de la celda que utiliza *BERT Embeddings* para la clasificación de textos, pues al estar poco familiarizados aún con estas herramientas, hemos recurrido a numerosas fuentes como IAs, *Google* o, especialmente *YouTube*, para comprender mejor su funcionamiento:

1. Carga del Modelo de *BERT*:
   - Se carga el modelo preentrenado de *BERT* y su tokenizador correspondiente utilizando la biblioteca “transformers”.
   - El modelo utilizado es “bert-base-uncased”, que es una versión de *BERT* entrenada en texto en minúsculas sin distinción de mayúsculas.

2. Generación de Embeddings:
   - Se define una función para generar embeddings de *BERT* a partir de los textos de entrada.
   - La función tokeniza los textos, los pasa a través del modelo de *BERT* y extrae los embeddings.
   - Los embeddings de cada token se promedian para obtener una representación vectorial del texto completo.

Tras la implementación de cada uno de los métodos, se encuentra una celda con la función “classify_subreddit(text)”, expresada para aplicar cada uno de los métodos de manera específica, que nos permite ver un ejemplo del funcionamiento de dichos métodos.

En base a los resultados obtenidos durante todas las implementaciones, pruebas, intentos y modificaciones, nos tenemos que decantar por el método basado en TF-IDF; dado que, bajo nuestras consideraciones y razonamientos expuestos durante todo este documento, es el método que mejor resultados nos ha brindado, pues este enfoque ha mostrado ser el más efectivo, considerando un equilibrio entre el tiempo de procesamiento, la precisión y la fiabilidad del modelo. Cabe destacar que, al profundizar un poco más en el tema de los embeddings, hemos observado que probablemente sea el método que mejor se pueda adaptar y el más complejo. Sin embargo, dada su complejidad, quizás no hemos logrado aprovecharlo al máximo, ya que nos ha resultado difícil comprender su funcionamiento a través de investigación y prueba y error. A pesar de esto, ciñéndonos a nuestros resultados, optamos por el método TF-IDF.

Por último, aclaramos que la explicación de cada uno de los pasos que se han ido siguiendo durante la implementación se encuentran en los comentarios del código.